

Inteligentne metody optymalizacji Optymalizacja efektywności lokalnego przeszukiwania

Wykorzystanie ocen ruchów z poprzednich iteracji i ruchów kandydackich w lokalnym przeszukiwaniu

1. Opis zadania

Celem zadania było zbadanie i rozwiązanie zmodyfikowanego problemu komiwojażera (TSP) z uwzględnieniem zysków wierzchołków. W klasycznym TSP celem jest znalezienie najkrótszego cyklu przechodzącego przez wszystkie wierzchołki. W naszym wariacie każdemu wierzchołkowi przypisany jest zysk, a zadanie polega na wyborze podzbioru wierzchołków oraz ułożeniu cyklu Hamiltona w taki sposób, aby maksymalizować funkcję celu:

$$F(S) = \sum_{v \in S} profit(v) - length(S)$$

gdzie S to wybrane wierzchołki cyklu, $profit(v)$ to zysk wierzchołka v , a $length(S)$ to długość cyklu

Problem ten wymaga znalezienia kompromisu między zyskiem z odwiedzonych wierzchołków a kosztem przemieszczania się między nimi. W praktyce oznacza to konieczność starannie przemyślanego wyboru wierzchołków i kolejności ich odwiedzenia, co czyni problem interesującym z punktu widzenia optymalizacji kombinatorycznej.

Głównym celem było zwiększenie efektywności czasowej lokalnego przeszukiwania w wersji stromej (*steepest local search*) przy zachowaniu możliwie wysokiej jakości uzyskiwanych rozwiązań. Jako punkt odniesienia wykorzystano pełne lokalne przeszukiwanie w wersji stromej z najlepszym sąsiedztwem wybranym w poprzednim zadaniu oraz najlepszą heurystykę konstrukcyjną z zadania 1. Wszystkie metody lokalnego przeszukiwania startowały z rozwiązań losowych.

Zaimplementowano dwa niezależne mechanizmy przyspieszające:

1. Listę ruchów poprawiających (Move List), wykorzystującą oceny ruchów z poprzednich iteracji.
2. Ruchy kandydackie (Candidate Moves), ograniczające przeglądane sąsiedztwo do ruchów związanych z najbliższymi wierzchołkami.

Dla ruchów kandydackich przetestowano kilka wartości parametru K , określającego liczbę najbliższych sąsiadów zapamiętywanych dla każdego wierzchołka.

2. Zaimplementowane algorytmy

Eksperyment przeprowadzono na dwóch instancjach testowych: A oraz B.

Każdy algorytm uruchomiono 100 razy dla każdej instancji. W przypadku lokalnego przeszukiwania rozwiązania startowe były generowane losowo.

Rozwiązania były sprawdzane narzędziem SolutionChecker, a najlepsze trasy zostały zwizualizowane w Pythonie z użyciem biblioteki matplotlib.

Dodatkowo wykonano eksperyment strojenia parametru K dla ruchów kandydackich, testując wartości: $K = 10$, $K = 30$, $K = 50$, $K = 100$

2.1. Regret2 - Heurystyka konstrukcyjna

```
used = tablica rozmiaru Instancja.n wypełniona Fałsz

start = losowy wierzchołek

Cykl = lista zawierająca start

used[start] = Prawda

// wybór drugiego wierzchołka

nxt = wierzchołek nieużyty o najmniejszej odległości od start

Dodaj nxt do Cykl

used[nxt] = Prawda

Dopóki rozmiar(Cykl) < Instancja.n:

    bestRegret = -∞

    bestV = brak

    Dla każdego wierzchołka v nieużytego:

        Incs = []
```

Dla każdej krawędzi (a,b) w Cykl:

```
inc = dist[a][v] + dist[v][b] - dist[a][b]
```

Dodaj inc do Incs

Posortuj Incs rosnąco

Jeśli liczba elementów Incs < 2:

Kontynuuj

```
regret = Incs[1] - Incs[0]
```

Jeśli używana jest wersja weighted:

```
regret = regret + w * (-Incs[0])
```

Jeśli regret > bestRegret:

```
bestRegret = regret
```

```
bestV = v
```

// wybór najlepszego miejsca wstawienia bestV

```
bestInc = +∞
```

```
bestPos = brak
```

Dla każdej krawędzi (a,b) w Cykl:

```
inc = dist[a][bestV] + dist[bestV][b] - dist[a][b]
```

Jeśli inc < bestInc:

```
bestInc = inc
```

```
bestPos = pozycja przed b
```

```

Wstaw bestV do Cykl na pozycji bestPos

used[bestV] = Prawda

Sol.cycle = Cykl

Oblicz statystyki Sol

// faza redukcji

Dopóki istnieje ruch REMOVE o dodatnim zysku i rozmiar(Cykl) > 2:

    Dla każdego wierzchołka v w Cykl:

        prev_v = poprzednik v w cyklu

        next_v = następnik v w cyklu

        DeltaRemove =

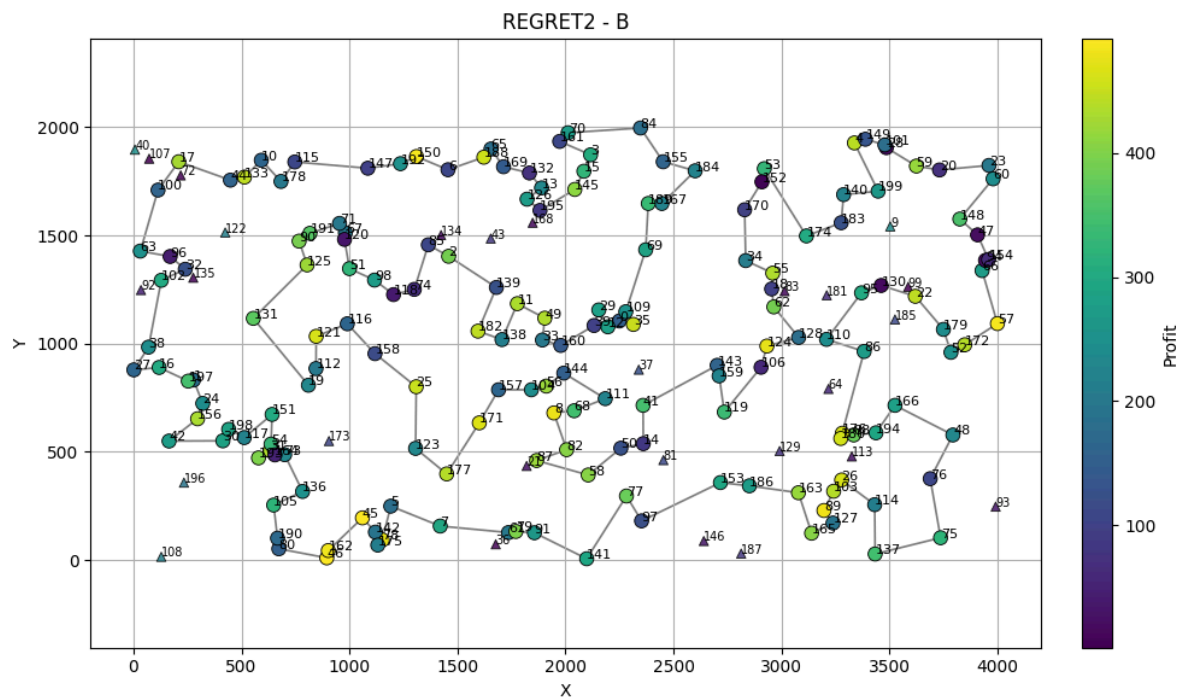
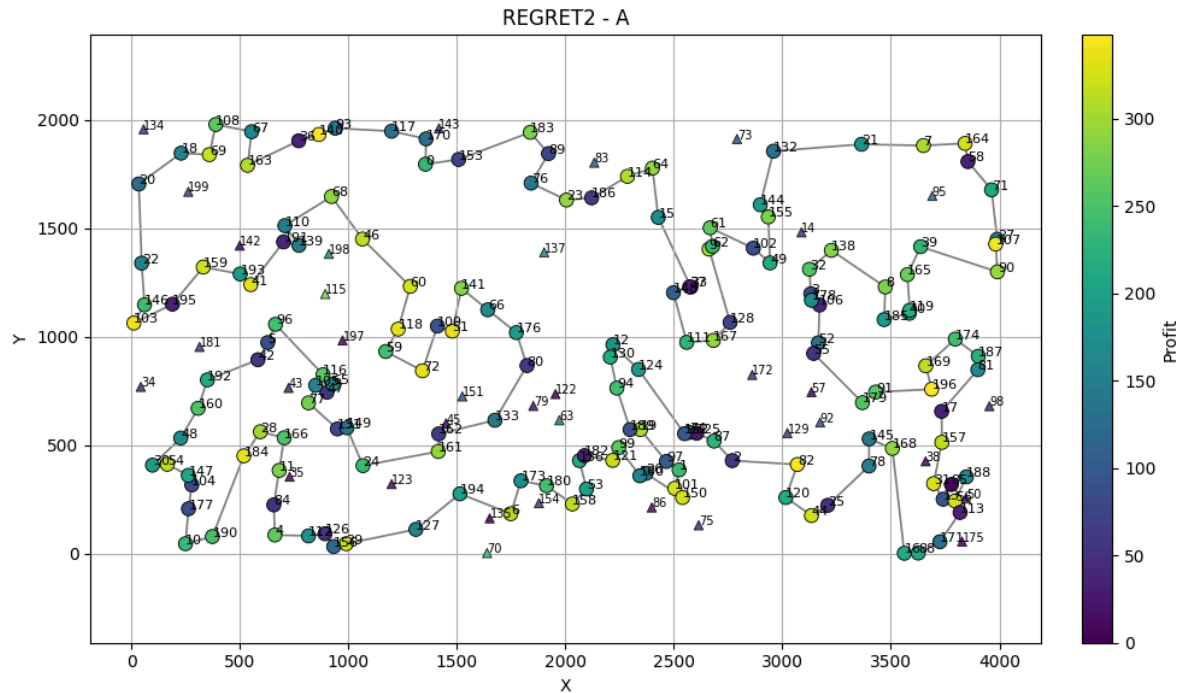
            (dist[prev_v][v] + dist[v][next_v] -
             dist[prev_v][next_v]) - Profit[v]

    Usuń wierzchołek o największym dodatnim DeltaRemove

Oblicz statystyki Sol

Zwróć Sol

```



2.2. Pełne lokalne przeszukiwanie strome EDGE SWAP

```
Sol = losowe rozwiązanie początkowe

Steepest(Sol, Instancja)

Oblicz statystyki Sol

Zwróć Sol
```

```

Improved = Prawda

Dopóki Improved == Prawda:

    Improved = Fałsz

    BestDelta = 0

    BestType = brak

    BestI = -1

    BestJ = -1

    BestV = -1

    BestPos = -1


    Cykl = Sol.cycle

    N = rozmiar(Cykl)

    inCycle = tablica rozmiaru Instancja.n wypełniona Fałsz

    Dla każdego v z Cykl:

        inCycle[v] = Prawda


    // RUCHY ADD

    Dla każdego wierzchołka v od 0 do Instancja.n - 1:

        Jeśli inCycle[v] == Prawda:

            Kontynuuj

        Dla każdego i od 0 do N - 1:

            j = (i + 1) MOD N


            DeltaAdd = Profit[v] - (dist[Cykl[i]][v]
                                    + dist[v][Cykl[j]] - dist[Cykl[i]][Cykl[j]])

```

```

        Jeśli DeltaAdd > BestDelta:

            BestDelta = DeltaAdd

            BestType = ADD

            BestV = v

            BestPos = j

// RUCHY REMOVE

Jeśli N > 2:

    Dla każdego idx od 0 do N - 1:

        prev_v = Cykl[(idx - 1 + N) MOD N]

        v = Cykl[idx]

        next_v = Cykl[(idx + 1) MOD N]

        DeltaRem = (dist[prev_v][v] + dist[v][next_v]
                    - dist[prev_v][next_v]) - Profit[v]

        Jeśli DeltaRem > BestDelta:

            BestDelta = DeltaRem

            BestType = REMOVE

            BestI = idx

// RUCHY EDGE_SWAP

Dla każdego i od 0 do N - 2:

    Dla każdego j od i + 2 do N - 1:

        Jeśli i == 0 ORAZ j == N - 1:

            Kontynuuj

```

```
a = Cykl[i]

b = Cykl[(i + 1) MOD N]

c = Cykl[j]

d = Cykl[(j + 1) MOD N]

DeltaSwap =

    (dist[a][b] + dist[c][d]) -

    (dist[a][c] + dist[b][d])
```

Jeśli $\Delta_{\text{Swap}} > \Delta_{\text{Best}}$:

```
BestDelta = DeltaSwap

BestType = EDGE_SWAP

BestI = i

BestJ = j
```

Jeśli $\Delta_{\text{Best}} > 0$:

```
Improved = Prawda
```

Jeśli $\text{BestType} == \text{ADD}$:

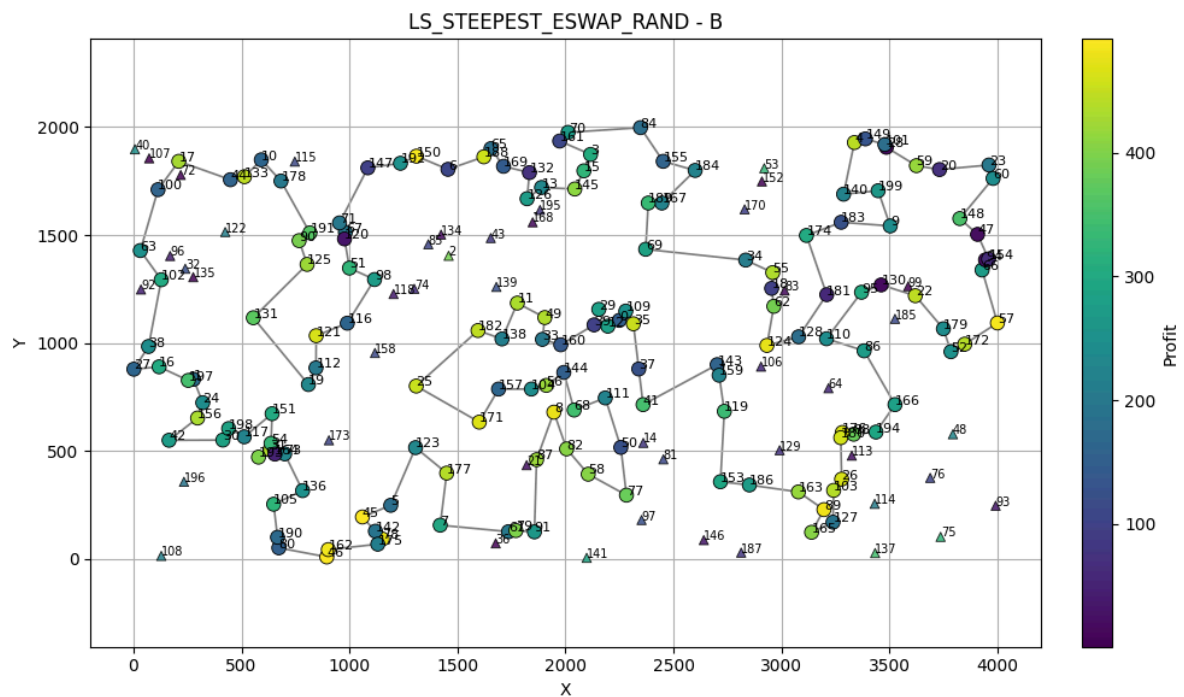
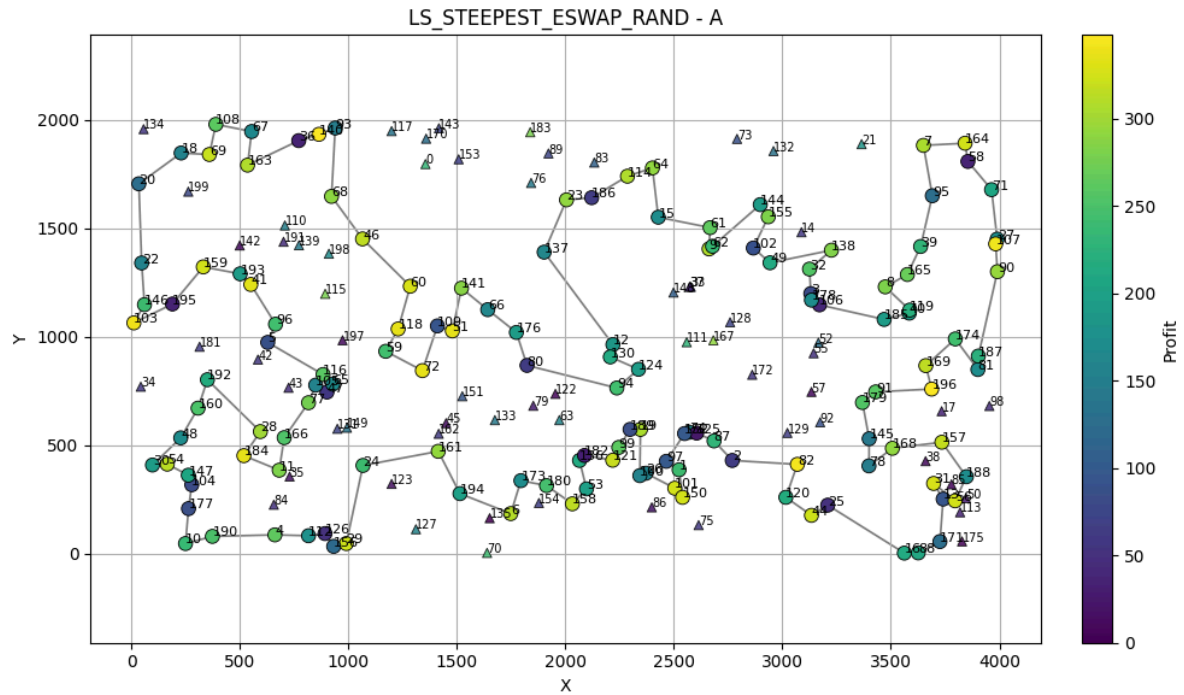
```
Wstaw BestV do Cykl na pozycji BestPos
```

Jeśli $\text{BestType} == \text{REMOVE}$:

```
Usuń wierzchołek z Cykl na pozycji BestI
```

Jeśli $\text{BestType} == \text{EDGE_SWAP}$:

```
Odwróć fragment Cykl od pozycji BestI + 1 do BestJ
```

2.3. Local Search with Move List

Struktury danych:

- MoveList – lista ruchów poprawiających,
uporządkowana rosnąco według delta
- Move – pojedynczy ruch:

```
- type  $\in$  {ADD, REMOVE, EDGE_SWAP}

- delta

- pola a, b, c, d

- next[v], prev[v], inCycle[v] – reprezentacja aktualnego cyklu

- startVertex – wierzchołek, od którego linearyzowany jest cykl
```

Interpretacja pól ruchu

```
- ADD: a = left, b = right, c = v

- REMOVE: a = left, b = v, c = right

- EDGE_SWAP: a = a, b = b, c = c, d = d
```

FUNKCJA Solve(Instancja)

Sol = losowe rozwiązanie początkowe

Jeśli Sol.cycle jest pusty:

Oblicz statystyki Sol

Zwróć Sol

Zbuduj reprezentację cyklu:

next[v], prev[v], inCycle[v]

startVertex = pierwszy wierzchołek cyklu

cycleSize = rozmiar cyklu

BuildImprovingMoveList(Instancja, bieżący cykl, MoveList)

Dopóki Prawda:

Applied = Fałsz

```
// przegląd listy ruchów od najlepszego

Dopóki MoveList nie jest pusta:

    Ruch = ostatni element MoveList

    Usuń ostatni element z MoveList

    Jeśli IsApplicable(Ruch, bieżący cykl) == Fałsz:

        Kontynuuj

    ApplyMove(Ruch, bieżący cykl)

    Applied = Prawda

    Przerwij

// jeżeli lista się "wyczerpała", zbuduj ją od nowa

Jeśli Applied == Fałsz:

    BuildImprovingMoveList(Instancja, bieżący cykl, MoveList)

    Jeśli MoveList jest pusta:

        Przerwij

Sol.cycle = MaterializeCycle(startVertex, next, cycleSize)

Oblicz statystyki Sol

Zwróć Sol
```

PROCEDURA BuildImprovingMoveList(Instancja, bieżący cykl, MoveList)

Wyczyść MoveList

Cykl = linearyzacja aktualnego cyklu

```
N = rozmiar(Cykl)
```

```
// ADD
```

Outside = lista wszystkich wierzchołków spoza cyklu

Dla każdej krawędzi (left,right) w Cykl:

Dla każdego v z Outside:

```
DeltaAdd = Profit[v] - (dist[left][v] + dist[v][right]
                        - dist[left][right])
```

Jeśli $\Delta_{Add} > 0$:

Dodaj ruch ADD do MoveList

// REMOVE

Jeśli $N > 2$:

Dla każdego wierzchołka v w Cykl:

```
left = poprzednik v
```

```
right = następnik v
```

```
DeltaRem = (dist[left][v] + dist[v][right]
            - dist[left][right]) - Profit[v]
```

```

        Jeśli DeltaRem > 0:

            Dodaj ruch REMOVE do MoveList

// EDGE_SWAP

Dla każdej dopuszczalnej pary krawędzi (a,b), (c,d):

    DeltaSwap = (dist[a][b] + dist[c][d]) -
                (dist[a][c] + dist[b][d])

    Jeśli DeltaSwap > 0:

        Dodaj ruch EDGE_SWAP do MoveList

Posortuj MoveList rosnąco według delta

FUNKCJA IsApplicable(Ruch, bieżący cykl)

    Jeśli Ruch.type == ADD:

        Zwróć Prawda, gdy:

            - wstawiany wierzchołek nadal jest poza cyklem

            - wskazana krawędź (left,right) nadal istnieje

    Jeśli Ruch.type == REMOVE:

        Zwróć Prawda, gdy:

            - usuwany wierzchołek nadal jest w cyklu

            - jego sąsiedzi są tacy sami jak w chwili oceny ruchu

    Jeśli Ruch.type == EDGE_SWAP:

        Jeśli obie usuwane krawędzie występują w zapamiętanym kierunku:

            Zwróć FORWARD

        Jeśli obie występują jednocześnie odwrócone:

```

Zwróć REVERSED

W przeciwnym wypadku:

Zwróć NOT_APPLICABLE

PROCEDURA ApplyMove(Ruch, bieżący cykl)

Jeśli Ruch.type == ADD:

Wstaw v pomiędzy left i right

cycleSize = cycleSize + 1

Jeśli Ruch.type == REMOVE:

Usuń v z cyklu

cycleSize = cycleSize - 1

Jeżeli v == startVertex:

startVertex = right

Jeśli Ruch.type == EDGE_SWAP:

Ustal, czy ruch jest FORWARD czy REVERSED

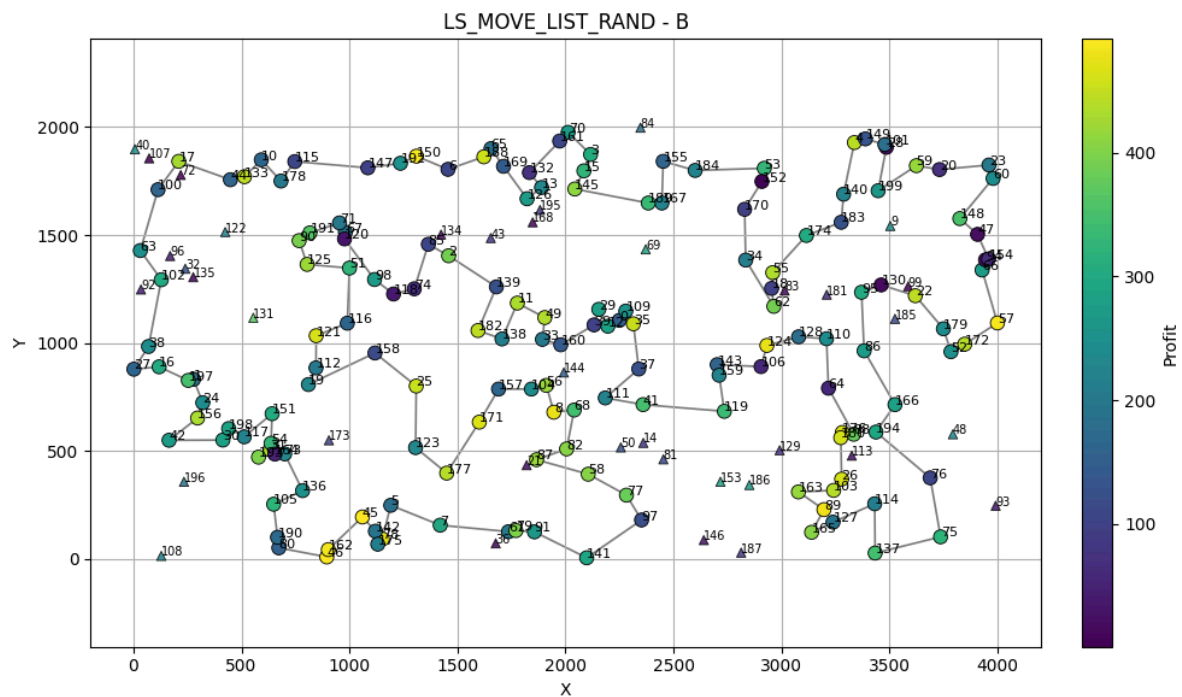
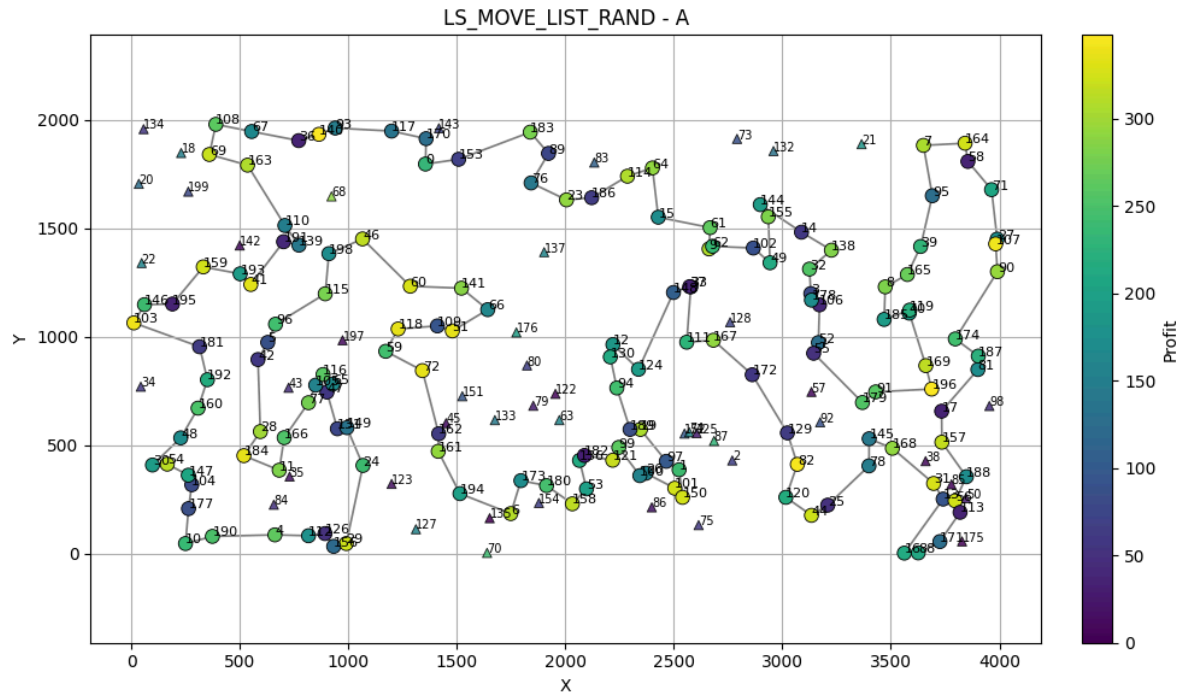
Odtwórz linearyzację cyklu

Wyznacz odpowiadający fragment do odwrócenia

ReverseSubpath(Cykl, start_rev, end_rev)

Odbuduj next i prev z nowej kolejności

startVertex = Cykl[0]



2.4. Local Search with Candidate Moves

Struktury danych:

- `Kandydaci[v]` – lista K najbliższych sąsiadów wierzchołka v
- `next[v]`, `prev[v]`, `inCycle[v]` – reprezentacja aktualnego cyklu

- Position[v] – pozycja wierzchołka w chwilowej linearyzacji cyklu
- MoveType $\in$ {NONE, INTRA_1, INTRA_2, ADD_NEXT, ADD_PREV, REMOVE}

FUNKCJA Solve(Instancja)

Jeśli listy kandydackie nie są policzone

lub ich rozmiar nie zgadza się z Instancja.n

lub zmieniła się liczba kandydatów:

PrecomputeCandidateMoves(Instancja, K)

Sol = losowe rozwiązanie początkowe

Jeśli Sol.cycle jest pusty:

Oblicz statystyki Sol

Zwróć Sol

Zbuduj reprezentację cyklu:

next[v], prev[v], inCycle[v]

startVertex = pierwszy wierzchołek cyklu

cycleSize = rozmiar cyklu

Position = tablica rozmiaru Instancja.n wypełniona -1

Improved = Prawda

Dopóki Improved == Prawda:

Improved = Fałsz

BestDelta = 0


```
BestType = NONE

BestI = -1

BestJorV = -1

Cykl = linearyzacja aktualnego cyklu

N = rozmiar(Cykl)

Dla każdego i od 0 do N - 1:

    Position[Cykl[i]] = i

// ruchy kandydackie

Dla każdego i od 0 do N - 1:

    n1 = Cykl[i]

    n1_next = Cykl[(i + 1) MOD N]

    n1_prev = Cykl[(i - 1 + N) MOD N]

    Dla każdego n2 z Kandydaci[n1]:

        Jeśli inCycle[n2] == Prawda:

            j = Position[n2]

            W przeciwnym wypadku:

                j = -1

        Jeśli j != -1:

            Jeśli i == j LUB i == (j + 1) MOD N
                LUB j == (i + 1) MOD N:
```

Kontynuuj

```
n2_next = Cykl[(j + 1) MOD N]
```

```
n2_prev = Cykl[(j - 1 + N) MOD N]
```

```
// INTRA_1
```

```
Delta1 =
```

```
(dist[n1][n1_next] + dist[n2_prev][n2]) -
```

```
(dist[n1][n2_prev] + dist[n1_next][n2])
```

```
Jeśli Delta1 > BestDelta:
```

```
zapamiętaj ruch INTRA_1
```

```
// INTRA_2
```

```
Delta2 =
```

```
(dist[n1_prev][n1] + dist[n2][n2_next]) -
```

```
(dist[n1_prev][n2] + dist[n1][n2_next])
```

```
Jeśli Delta2 > BestDelta:
```

```
zapamiętaj ruch INTRA_2
```

W przeciwnym wypadku:

```
// ADD_NEXT
```

```
DeltaAddNext = Profit[n2] - (dist[n1][n2]  
+ dist[n2][n1_next] - dist[n1][n1_next])
```

```

        Jeśli DeltaAddNext > BestDelta:

            zapamiętaj ruch ADD_NEXT

        // ADD_PREV

        DeltaAddPrev = Profit[n2] - (dist[n1_prev][n2]
            + dist[n2][n1] - dist[n1_prev][n1])

        Jeśli DeltaAddPrev > BestDelta:

            zapamiętaj ruch ADD_PREV

// REMOVE

Jeśli N > 2:

    Dla każdego idx od 0 do N - 1:

        v = Cykl[idx]

        prev_v = Cykl[(idx - 1 + N) MOD N]

        next_v = Cykl[(idx + 1) MOD N]

        DeltaRem = (dist[prev_v][v] + dist[v][next_v]
            - dist[prev_v][next_v]) - Profit[v]

        Jeśli DeltaRem > BestDelta:

            zapamiętaj ruch REMOVE

Jeśli BestDelta > 0:

    Improved = Prawda

```

```
Jeśli BestType == INTRA_1:

    ReverseSubpath(Cykl, BestI + 1, BestJorV - 1)

    Odbuduj next i prev z nowej kolejności

    startVertex = Cykl[0]

Jeśli BestType == INTRA_2:

    ReverseSubpath(Cykl, BestI, BestJorV)

    Odbuduj next i prev z nowej kolejności

    startVertex = Cykl[0]

Jeśli BestType == ADD_NEXT:

    Wstaw nowy wierzchołek za Cykl[BestI]

    cycleSize = cycleSize + 1

Jeśli BestType == ADD_PREV:

    Wstaw nowy wierzchołek przed Cykl[BestI]

    cycleSize = cycleSize + 1

    Jeśli Cykl[BestI] == startVertex:

        startVertex = nowy wierzchołek

Jeśli BestType == REMOVE:

    Usuń wskazany wierzchołek z cyklu

    cycleSize = cycleSize - 1

    Jeśli usunięto startVertex:

        startVertex = jego następnik
```

```
Sol.cycle = MaterializeCycle(startVertex, next, cycleSize)
```

```
Oblicz statystyki Sol
```

```
Zwróć Sol
```

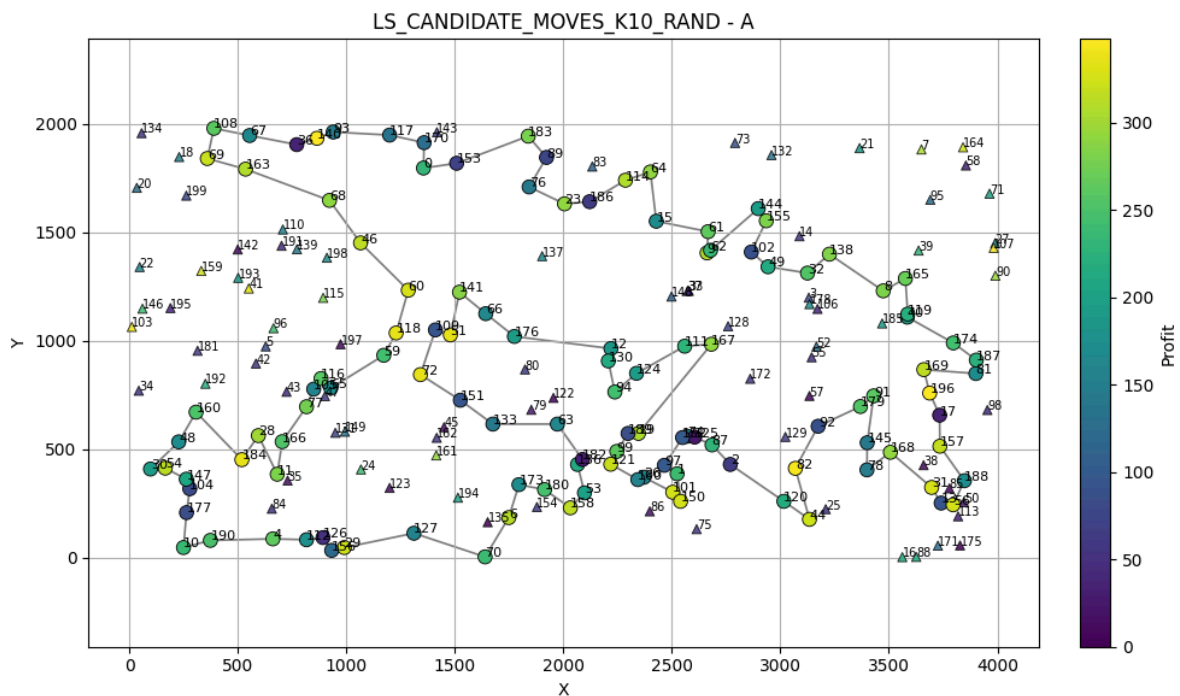
```
PROCEDURA PrecomputeCandidateMoves(Instancja, K)
```

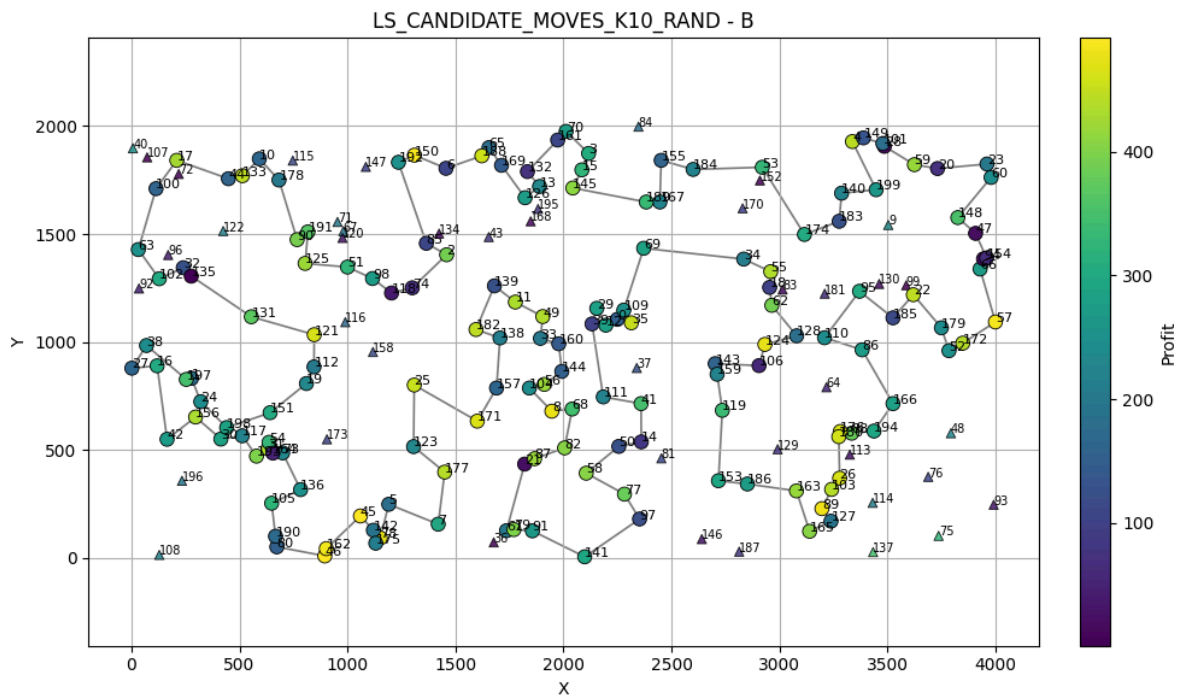
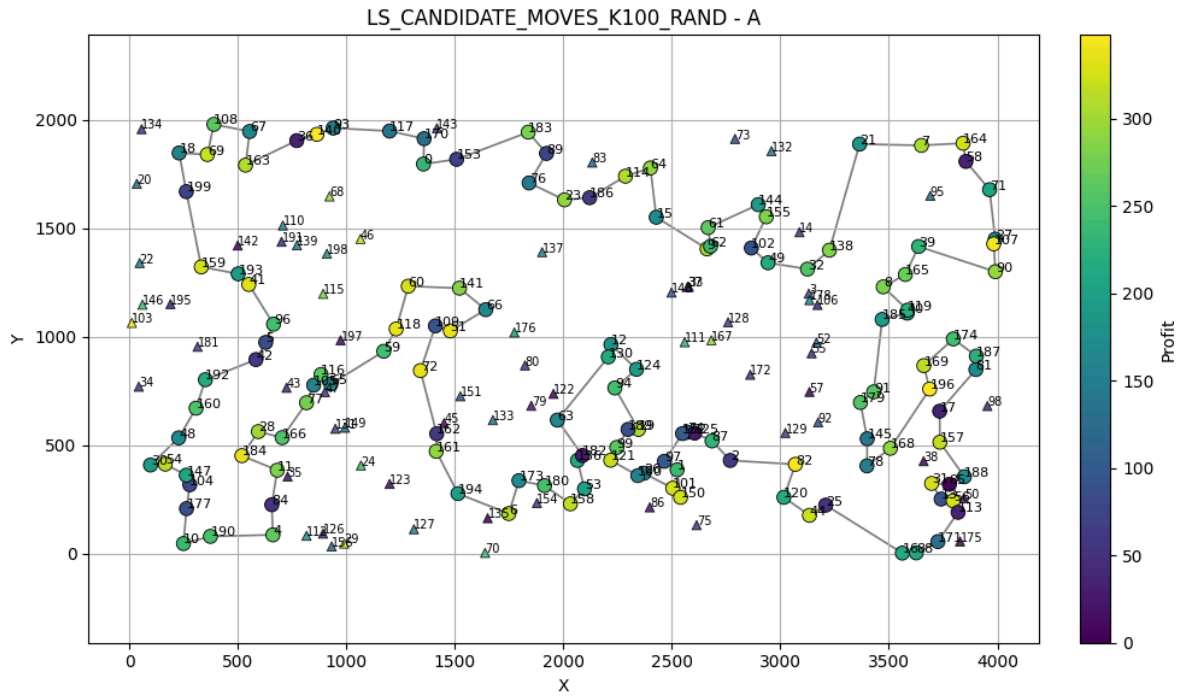
```
Dla każdego wierzchołka n1:
```

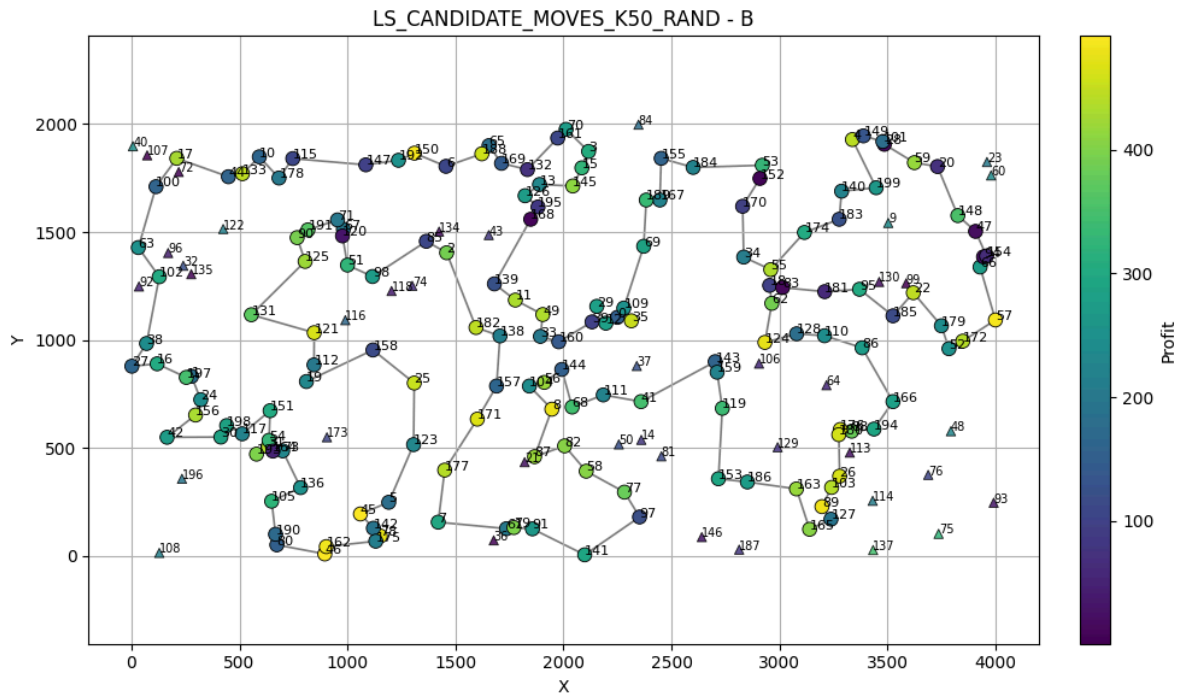
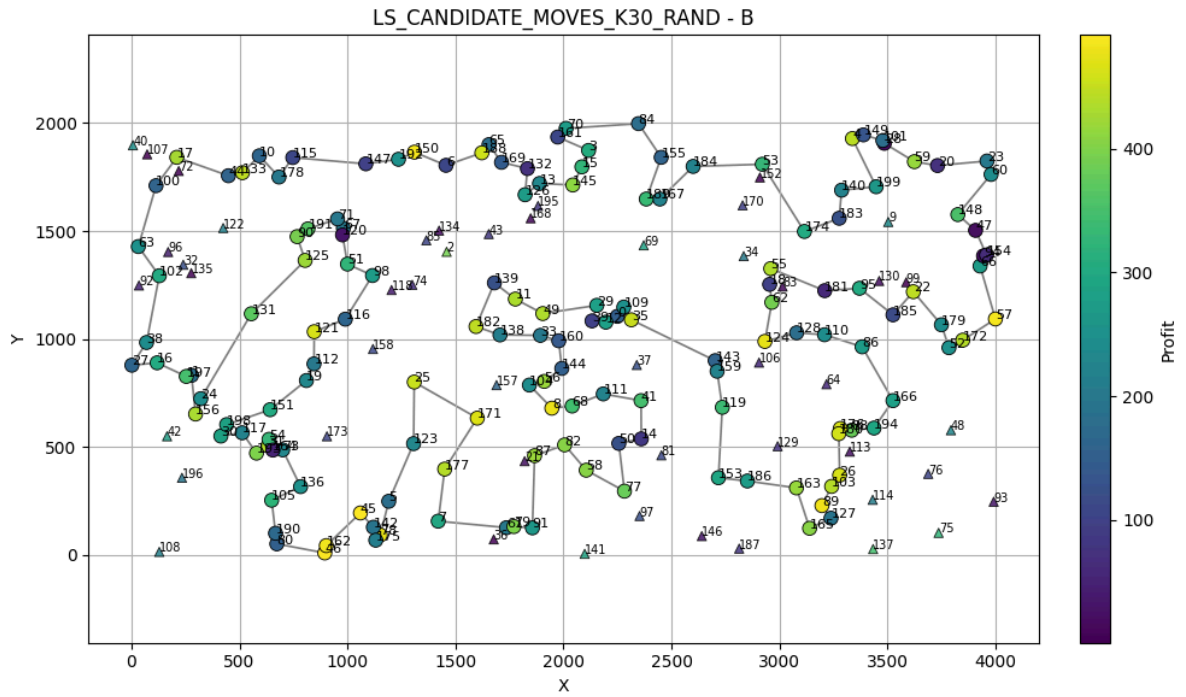
```
distances = lista wszystkich par (dist[n1][n2], n2),  
gdzie n2 ≠ n1
```

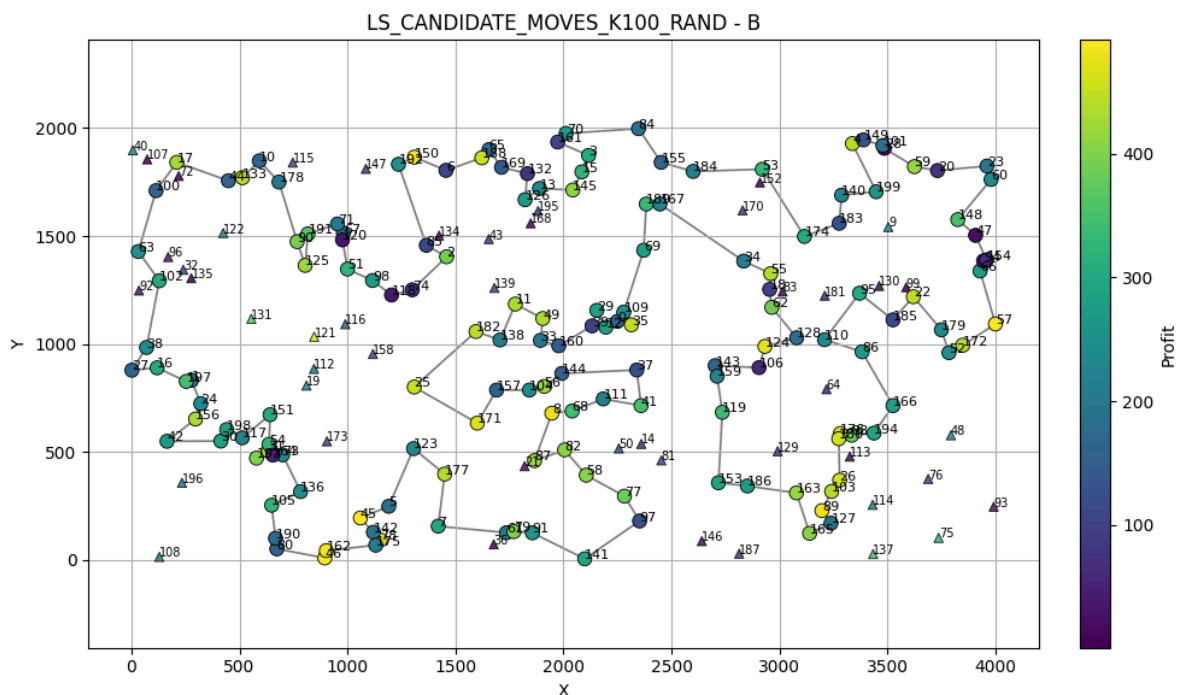
```
Posortuj distances rosnąco
```

```
Kandydaci[n1] = pierwsze K wierzchołków z distances
```









3. Wyniki eksperymentu obliczeniowego

Wartość funkcji celu

Metoda	Instancja A (średnia min–max)	Instancja B (średnia min–max)
REGRET2	6117.69 (4928 – 6934)	18051.9 (16877 – 19517)
LS_STEEPEST_ESWAP_RAND	6361.23 (4917 – 7982)	18016.4 (16253 – 19371)
LS_MOVE_LIST_RAND	6128.61 (4799 – 7727)	17660.8 (15740 – 19016)
LS_CANDIDATE_MOVES_K10_RAND	4768.79 (2040 – 6677)	16147.1 (10620 – 19302)
LS_CANDIDATE_MOVES_K30_RAND	5814.61 (3838 – 7692)	17209.2 (12789 – 19588)
LS_CANDIDATE_MOVES_K50_RAND	5988.34 (4305 – 7138)	17654.2 (14832 – 19556)
LS_CANDIDATE_MOVES_K100_RAND	6229.36 (4269 – 7728)	17896.4 (15457 – 19497)

Czas pracy (wraz z wygenerowaniem rozwiązania startowego)

Metoda	Instancja A (średnia min–max) [ms]	Instancja B (średnia min–max) [ms]
REGRET2	157.387 (154.011 – 166.144)	158.531 (154.458 – 167.975)
LS_STEEPEST_ESWAP RAND	34.3123 (28.9633 – 42.5192)	39.5021 (32.0823 – 55.6118)
LS_MOVE_LIST RAND	3.69234 (3.00669 – 5.23944)	3.87705 (3.00549 – 5.87385)
LS_CANDIDATE_MOVES_K10 RAND	3.18526 (1.91946 – 8.68161)	4.65718 (2.76359 – 8.89414)
LS_CANDIDATE_MOVES_K30 RAND	10.5335 (7.71054 – 15.5233)	14.235 (10.5385 – 21.6706)
LS_CANDIDATE_MOVES_K50 RAND	19.6646 (14.2212 – 29.0359)	24.9815 (19.2391 – 33.1907)
LS_CANDIDATE_MOVES_K100 RAND	44.0402 (35.3225 – 55.5467)	54.5002 (45.4885 – 61.8526)

4. Wnioski

Uzyskane wyniki pokazują, że pełne lokalne przeszukiwanie strome (LS_STEEPEST_ESWAP RAND) dawało najwyższą średnią jakość rozwiązań na instancji A oraz wyniki bardzo zbliżone do najlepszych na instancji B. Jednocześnie jego czas działania był wyraźnie większy niż w przypadku metod przyspieszonych.

Algorytm LS_MOVE_LIST RAND okazał się zdecydowanie najszybszą metodą lokalnego przeszukiwania spośród porównywanych wariantów. Na obu instancjach jego średni czas działania był około dziesięciokrotnie krótszy od pełnego lokalnego przeszukiwania stromej wersji, przy umiarkowanym pogorszeniu jakości rozwiązań. Oznacza to, że wykorzystanie listy wcześniej ocenionych ruchów istotnie poprawia efektywność czasową algorytmu.

W przypadku ruchów kandydackich zaobserwowano wyraźny kompromis pomiędzy czasem działania a jakością rozwiązań. Dla małych wartości parametru K algorytm działał bardzo szybko, ale uzyskiwane wartości funkcji celu były istotnie słabsze. Zwiększanie liczby kandydatów poprawiało jakość rozwiązań, jednak jednocześnie prowadziło do wzrostu czasu obliczeń. Dla $K = 100$ jakość rozwiązań była już zbliżona do pełnego lokalnego przeszukiwania, lecz czas działania przekraczał czas klasycznego LS_STEEPEST_ESWAP RAND, co oznacza, że tak duża liczba kandydatów nie dawała już praktycznej korzyści.

Najlepszy kompromis pomiędzy czasem a jakością wśród testowanych wariantów ruchów kandydackich uzyskano dla $K = 50$. Wariant ten był wyraźnie szybszy od pełnego lokalnego przeszukiwania, a jednocześnie dawał rozwiązania o jakości relatywnie bliskiej rozwiązaniom uzyskiwanym przez pełną wersję stromej heurystyki.

Na podstawie przeprowadzonych eksperymentów można stwierdzić, że mechanizm Move List okazał się najbardziej efektywnym sposobem przyspieszenia lokalnego przeszukiwania w badanym problemie. Mechanizm Candidate Moves również pozwalał skrócić czas działania, lecz jego skuteczność zależała silnie od doboru parametru K oraz początkowego rozwiązania losowego.

Kod źródłowy:

https://github.com/KamieniarzJakub/155845_155945/tree/master/zadanie1